Práctica 4

1)ALGORITMO DE PRIM

Implementar un módulo Prim.py, de forma que, tras meter el nombre de un fichero, calcule y escriba por pantalla la solución óptima. Razone convenientemente la complejidad temporal del algoritmo implementado

La complejidad del algoritmo de Prim será de O(n^2), básicamente, esto se debe a que estamos trabajando con matrices de adyacencias y, por ende, al tener que recorrer estas, vamos a vernos forzados a tener una complejidad mínima de O(n^2).

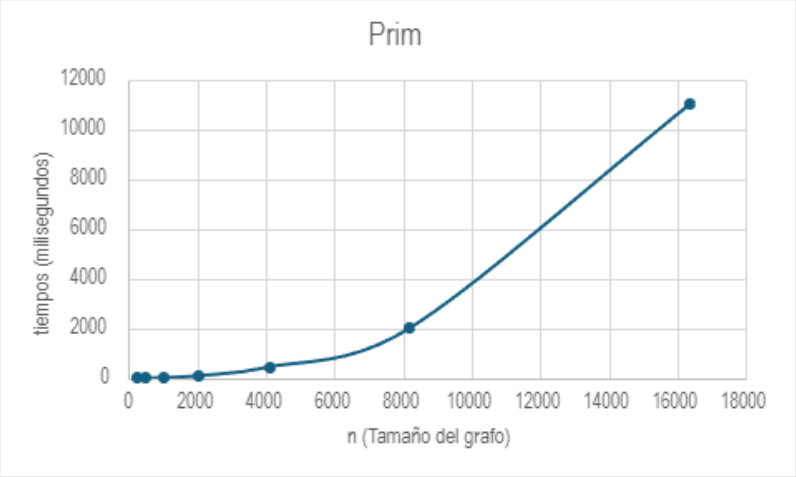
En cuanto a si la implementación del método es coherente con esto, contamos con un bucle principal (de complejidad O(n) pues recorrer en función del tamaño del grafo sobre el que realizamos el algoritmo) dentro del cual se ejecutan otros (todos de la misma complejidad que el principal), pero sin llegar a estar estos anidados, por lo que la complejidad resultante, al ser el número de máximo de bucles anidados es, efectivamente O(n^2).

Implementar un módulo PrimTiempos.py, que vaya creando grafos aleatorios de diversos tamaños y calculando el tiempo que tarda el algoritmo hecho en el apartado anterior en resolver el problema, de forma que se pueda ir rellenando la siguiente tabla. Razone: ¿siguen la complejidad calculada los tiempos de la tabla?

Para que se cumpla la complejidad establecida, los tiempos obtenidos deberían ir multiplicándose por 4, aproximadamente.

Esto medido con un procesador Intel Core i5 12400 2.5GHz

n	t Prim
256	2
512	8
1024	28
2048	109
4096	458
8192	2021
16384	11069
32768	ERROR



Como se puede apreciar, este incremento parece cumplirse hasta una vez se llega a n= 16384, en el cual en líneas generales según he ejecutado más veces suele aumentar más de lo esperado. Y en valores de n posteriores a este, sencillamente salta una excepción en Java de "*OutOfMemoryError",* por lo que yo intuyo que ocurrirá al estar creando grafos y vectores tan grandes en cada iteración.
No sé hasta qué punto esto es lo esperado, pero a mi código no le he podido encontrar errores que resuelvan este problema.